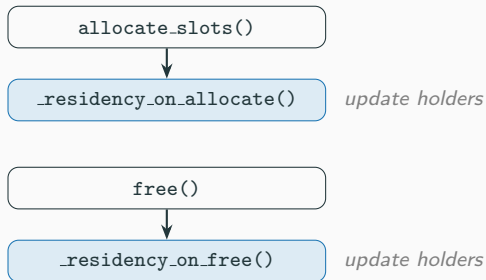


Slide 1: Instrumentation Architecture

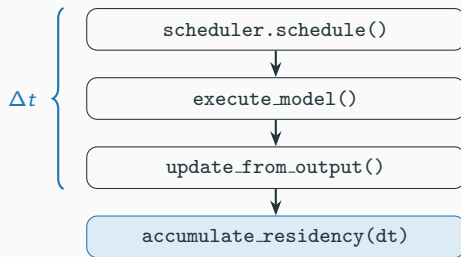
Incremental tracking



State lives on KVCacheManager.

Updated only when blocks change ownership.

Per-step charging: $O(N_{\text{tenants}})$



Reads `tenant_resident_tokens` dict.

No block iteration at step time.

Slide 2: Implementation Pseudocode

At allocation/free time (amortized):

```
# On allocate block B for tenant T:
if B not in holders:
    holders[B] = {T: 1}
    tenant_tokens[T] += block_size
elif T not in holders[B]:
    k = len(holders[B]) # cross-tenant sharing
    for X in holders[B]:
        tenant_tokens[X] -= block_size/k
        tenant_tokens[X] += block_size/(k+1)
    holders[B][T] = 1
    tenant_tokens[T] += block_size / (k+1)
else:
    holders[B][T] += 1 # intra-tenant
# On free block B for tenant T:
holders[B][T] -= 1
if holders[B][T] == 0:
    k = len(holders[B])
    del holders[B][T]
    tenant_tokens[T] -= block_size / k
    for X in holders[B]:
        tenant_tokens[X] += block_size/(k-1)
        - block_size/k
```

At step time — $O(N_{\text{tenants}})$:

```
def step():
    t0 = time.monotonic()
    schedule() # allocates blocks
    execute_model() # GPU forward pass
    update_from_output() # frees completed reqs
    dt = time.monotonic() - t0

    # Charge all resident memory:
    for tenant, tokens in tenant_tokens.items():
        residency_counter.labels(
            tenant_id=tenant).inc(tokens * dt)
```

Conservation invariant:

```
# Always holds:
sum(tenant_tokens.values())
== total_allocated_blocks * block_size
```

All GPU memory-time is fully accounted for. No over- or under-counting.

Slide 3: Workload Design

Experiment setup

- 1 model (e.g. Llama-3.1-8B), 1 GPU
- N tenants ($\sim 3-5$), each sends identical workload
- Same prompt length, output length, arrival rate
- Unique random prompts per request (no prefix cache hits)
- Prefix caching enabled but never triggers

What this tests

- Does the metric work end-to-end?
- Under symmetric load, is residency equal across tenants?
- Does the Prometheus counter accumulate correctly over time?

Configuration

Parameter	Value
Tenants	3
Prompt length	1024 tokens
Max output tokens	128
Rate per tenant	2 req/s (Poisson)
Duration	5 min (300 reqs/tenant)
Model	Llama-3.1-8B

Expected result

- Symmetric load $\rightarrow$ expect $\approx$ equal residency
- Any skew reveals scheduling unfairness (FCFS, batching, preemption)
- Conservation: $\sum \text{rate}(\text{residency}) = \text{total KV blocks} \times \text{block_size}$